

Reviewer Pack — Sandpile-Group Order of Lifted IP Bipartite Graph Bounds Dec...

Entry 69ad4d50946e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 07:15:12 UTC

Sandpile-Group Order of Lifted IP Bipartite Graph Bounds Decision Tree Depth

Entry ID: 69ad4d50946e **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-27 07:15:12 UTC

1. Conjecture

Field A (mathematical branch): Algebraic graph theory via the critical (sandpile) group $K(G)$ and Kirchhoff's Matrix-Tree theorem applied to bipartite communication graphs (rare in CS lower bounds; cf. Bacher-de la Harpe-Nagnibeda, Biggs) **Field B** (complexity object): Deterministic communication complexity of Raz-McKenzie / Göös-Pitassi-style lifted Boolean functions $f \circ \text{IP}_b^n$ with the inner-product gadget

Statement:

Let $f: \{0,1\}^n \rightarrow \{0,1\}$ have exact decision-tree depth $d=D(f)$, let $b \in \{2,3\}$ be the IP-gadget block size, and let $M(f,b) \in \{0,1\}^{\{2\}^{bn} \times 2^{\{bn\}}}$ be the lifted communication matrix with $M[x,y] = f((x_1, y_1), \dots, (x_n, y_n))$ over $\text{GF}(2)$. Form the bipartite graph $G(f,b)$ with rows on one side, columns on the other, and an edge exactly where $M[x,y]=1$; let $\tau(G)$ denote its spanning-tree count (Kirchhoff determinant of any reduced Laplacian, with $\tau=0$ if disconnected and τ a product over components otherwise). Conjecture: for every non-constant f with $n \leq 3$ and $b \in \{2,3\}$, $\log_2 \tau(G(f,b)) \geq d \cdot 2^{\{bn-2\}}$, and equality is achieved (within $\pm b$) precisely when f is the parity (XOR_n) function.

Rationale (proposer's reasoning):

Lifting overhead per block in deterministic IP-lifting morally measures how many distinct rectangles each query forces; the sandpile group order is the lattice index of the cycle/cut spaces of G and counts admissible flow/charge configurations on G 's bipartite structure. Linking $\tau(G)$ to $D(f) \cdot (\text{gadget block area})$ plants a query-to-communication bridge through pure graph algebra, sidestepping algebrization since spanning-tree counting is not a relativizing arithmetic primitive. Tightness on parity is suggested because XOR-lifted matrices are Hadamard-like, giving maximally regular Laplacian spectra and hence nearly extremal spanning-tree counts.

Taxonomy category: LIFTING (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f90b33c3075b08f3

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across all tested (n,b,f) with $n \in \{1,2,3\}$, $b \in \{2,3\}$, enumerating all $2^{\binom{2}{n}}$ truth tables for $n \leq 2$ and 200 random non-constant truth tables for $n=3$ (one fixed seed per n), the conjecture is SUPPORTED iff every non-constant f satisfies $\log_2 \tau(G(f,b)) \geq d \cdot 2^{\{bn-2\}}$ (slack ≥ 0) AND for $f=\text{XOR}_n$ the slack lies in $[0,b]$. A single violation falsifies.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.86	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.78	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - sandpile group bipartite graph communication complexity lifting - critical group spanning trees lifted Boolean function inner product gadget - Kirchhoff matrix-tree theorem decision tree depth Raz-McKenzie simulation

Top relevant hits considered: - [<http://arxiv.org/abs/1906.08515v1>] An overview on the bipartite divisor graph for the set of irreducible character degrees - [<http://arxiv.org/abs/1705.07519v1>] Sandpile Groups of Random Bipartite Graphs - [<http://arxiv.org/abs/1904.12209v2>] Sandpile monomorphisms and limits - [<http://arxiv.org/abs/1012.2478v2>] Packing spanning trees and the k -tree protocol - [<http://arxiv.org/abs/0904.3087v4>] Distributed Maintenance of Anytime Available Spanning Trees in Dynamic Networks - [<http://arxiv.org/abs/hep-ph/0610012v1>] Tevatron-for-LHC Report of the QCD Working Group - [<http://arxiv.org/abs/2007.09501v3>] A family of matrix-tree multijections - [<http://arxiv.org/abs/2512.11833v1>] Soft Decision Tree classifier: explainable and extendable PyTorch implementation

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
import json
from itertools import product

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def memoized_recursive_query_enumeration(f, n):
        @lru_cache(None)
        def query(x):
            if x == 0:
                return 1
```

```

        else:
            return sum(query((x >> i) ^ (x & -x)) for i in range(n)) % 2
    return query

def build_lifted_communication_matrix(f, n, b):
    M = [[f(tuple(x[i:i+b] for i in range(0, len(x), b))) for x in product([0, 1], repeat=n)]]
    return M

def build_bipartite_graph_laplacian(M):
    n = int(math.log2(len(M)))
    L = [[0] * (n * b) for _ in range(n * b)]
    for i in range(n * b):
        for j in range(n * b):
            if M[i // b][j // b] == 1:
                L[i][j] = -1
            if i != j:
                L[j][i] = 1
    return L

def determinant(M, mod=604853):
    n = len(M)
    det = 1
    for i in range(n):
        pivot = M[i][i]
        for j in range(i + 1, n):
            factor = M[j][i] * pow(pivot, -1, mod) % mod
            for k in range(i, n):
                M[j][k] = (M[j][k] - factor * M[i][k]) % mod
        det = det * pivot % mod
    return det

def fraction_free_bareiss(M):
    n = len(M)
    A = [[0] * (n + 1) for _ in range(n + 1)]
    for i, row in enumerate(M):
        A[i][i] = 1
        for j, val in enumerate(row):
            A[i][j + n + 1] = val
    det = 0
    for k in range(1, n + 1):
        pivot = A[k - 1][k - 1]
        for i in range(k, n + 1):
            for j in range(k, n + 1):
                A[i][j] = (A[i][j] * pivot - A[i - 1][j] * A[k - 1][i]) % mod
        det = A[n][n]
    return det

def spanning_tree_count(L):
    n = len(L)
    L_mod_2 = [(x + y) % 2 for x, y in zip(row[:n], row[n:])] for row in L
    det = fraction_free_bareiss(L_mod_2)
    if det == 0:
        return 0
    else:
        return det

```

```

def parity_function(n):
    return lambda x: sum(x) % 2

n_values = [1, 2, 3]
b_values = [2, 3]
results = []

for n in n_values:
    for b in b_values:
        if n <= 2:
            f_values = list(product([0, 1], repeat=2**n))
        else:
            f_values = random.sample(list(product([0, 1], repeat=2**n)), 200)

        for f in f_values:
            query = memoized_recursive_query_enumeration(lambda x: f[x], n)
            d = query(0)

            M = build_lifted_communication_matrix(f, n, b)
            L = build_bipartite_graph_laplacian(M)
            tau = spanning_tree_count(L)

            if tau == 0:
                continue

            log_tau = math.log2(tau)
            lower_bound = d * 2**(b*n - 2)
            slack = abs(log_tau - lower_bound)

            result = {
                "metric_name": "log2_tau",
                "metric_value": log_tau,
                "instances_tested": 1,
                "conjecture_holds": slack <= b and (slack == 0 or f != parity_function(n)),
                "counterexample": "" if slack <= b else f"n={n}, b={b}"
            }
            results.append(result)

mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

return {
    "seed": seed,
    "mean_metric_value": mean_metric_value,
    "std_metric_value": std_metric_value,
    "support_fraction": support_fraction
}

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [11, 23, 37, 53, 71]

    for seed in seeds:
        result = run_trial(seed)
        print(json.dumps({"TRIAL": result}))

```

```

mean_metric_value = sum(result["mean_metric_value"] for result in results) / len(results)
std_metric_value = math.sqrt(sum((result["mean_metric_value"] - mean_metric_value)**2 for result in results) / len(results))
support_fraction = sum(1 for result in results if result["support_fraction"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"n={result['counterexample']}\", first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fb3bd595.py", line 134, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fb3bd595.py", line 96, in run_trial
    query = memoized_recursive_query_enumeration(lambda x: f[x], n)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fb3bd595.py", line 21, in memoized_recursive_query_enumeration
    @lru_cache(None)
    ~~~~~^
NameError: name 'lru_cache' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a NameError ('lru_cache' is not defined) before producing any data, so no support fraction or slack values were computed. Per the rules, a crashed test yields INCONCLUSIVE. | next: Fix the test by importing lru_cache from functools (add 'from functools import lru_cache') and rerun the full enumeration over $n \in \{1, 2, 3\}$, $b \in \{2, 3\}$.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	167986
2	preregistration	claude_max	opus	0	0	5898
3	novelty	claude_max	opus	0	0	3530
4	novelty	claude_max	opus	0	0	10064
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18377
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15769
7	judge	claude_max	opus	0	0	4033

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 225656 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/69ad4d50946e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/69ad4d50946e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/69ad4d50946e.tar.gz` (if generated)